



Reconfigurable content-addressable memory (CAM) on FPGAs: A tutorial and survey

Muhammad Irfan^{a,b,*}, Abdurrashid Ibrahim Sanka^a, Zahid Ullah^c, Ray C.C. Cheung^a

^a Department of Electrical Engineering, City University of Hong Kong, Hong Kong

^b Faculty of Electrical Engineering, Ghulam Ishaq Khan Institute of Engineering Sciences and Technology, Topi, Pakistan

^c Pak-Austria Fachhochschule: Institute of Applied Sciences & Technology, Haripur, Pakistan

ARTICLE INFO

Article history:

Received 5 May 2021

Received in revised form 19 September 2021

Accepted 27 September 2021

Available online 8 October 2021

Keywords:

Field-programmable gate arrays

Content-addressable memory

Reconfigurable computing

Network router

Random-access memory

SRAM-based TCAM

ABSTRACT

Content-addressable memory (CAM) is a massively parallel searching device that returns the address of a given search input in one clock cycle. Field-programmable gate array (FPGA)-based CAMs are becoming popular due to their applications in the latest networking systems, e.g., software-defined networks (SDNs) leading to upcoming 5G networks. Ternary CAM (TCAM) implements a routing table in a network router to classify and forward data packets where don't care bits (X-bits) correspond to multiple addresses. FPGAs do not have a hard-core CAM, although it is a prime element in networking applications. This paper serves as a comprehensive survey on FPGA-based CAM/TCAMs implemented using block random-access memory (BRAM), lookup table RAM (LUTRAM), and flip-flops (FFs). BRAM-based TCAM suffers from the pre-processing of mapping data, requires the data to be in a specific order in some cases, and has a large SRAM/TCAM bit ratio. LUTRAM-based CAM/TCAM suffers from wide bit-wise ANDing, high routing complexity, but has a small SRAM/TCAM bit ratio of 14 compared to 16 in the case of BRAM-based TCAM. Shallow and wide RAM blocks are required to implement large-size RAM-based TCAMs (BRAM-based and LUTRAM-based TCAMs). FF-based TCAMs use FFs as their memory elements and have reduced hardware costs per TCAM bit. However, due to the routing complexity, it suffers from scalability and a large amount of power consumption. The update latency of BRAM-based TCAM and LUTRAM-based TCAM is proportional to the depth of BRAM and LUTRAM, respectively. However, FF-based CAM updates in 1 or 2 clock cycles depending on the availability of input/output pins on target FPGA.

© 2021 Published by Elsevier B.V.

1. Introduction

The idea of content-based memory comes from the biological brain where the stored data is accessed irrespective of its location [1–3]. Typical computer memory, random-access memory (RAM) which is also known as address-based memory, requires the data to be indexed. The index (address) is required to access or store data in a computer memory [4]. In contrast, the human brain stores information and does not need any index to retrieve the information [5]. The content of the information itself or a portion of it is used to recall the information from the human brain, and this type of memory is known as content-addressable memory (CAM) [6].

Dudley Buck, in the 1950s, proposed a parallel computation circuitry named “recognition unit”, which was later realized to

be a content-addressable storage [7]. Fig. 1 shows a simplified CAM that takes the input as a search key and compares it with all of the rules in parallel to produce a matchline for each location. A priority encoder (PE) converts the matchlines into an address (Addr) where the search key matches the stored rule.

CAM tremendously boosts searching speed, however, it is limited in size, consumes significant power, and expensive on hardware. CAM is used for routing tables in a network, such as software-defined networks (SDN) to retrieve the next-hop and meet the high bandwidth of the systems [8–12]. SDNs are considered to be the future ultra-high bandwidth networks, and their key feature is “reconfigurability”, which is best provided by the modern reconfigurable hardware, i.e., field-programmable gate arrays (FPGAs) [13–15].

CAM is becoming an essential part of many embedded applications, such as SDNs, but is missing in modern FPGAs. FPGAs have a vast range of applications besides networking [16] however, FPGA vendors have not yet included a hard-core CAM on modern FPGA devices [17,18]. Besides, to support networking applications, such as SDN, the vendors have provided soft intellectual property

* Corresponding author.

E-mail addresses: m.irfan@iee.edu.hk (M. Irfan), iasanka2-c@my.cityu.edu.hk (A.I. Sanka), zahid.ullah@fecid.paf-iast.edu.pk (Z. Ullah), r.cheung@cityu.edu.hk (R.C.C. Cheung).

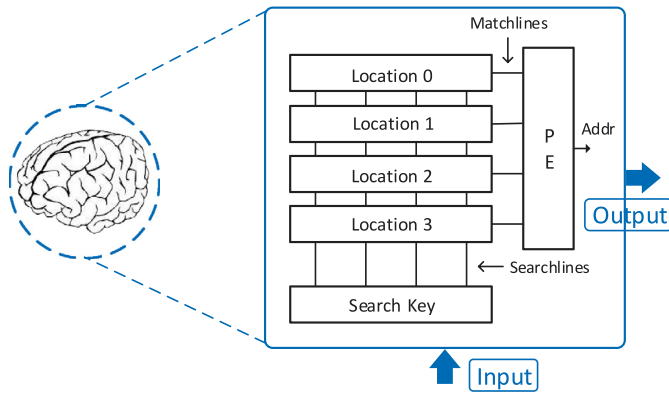


Fig. 1. Brain and a simplified content-addressable memory. (PE: Priority encoder, Addr: Address of the location where the input search key matches). [This figure shows a comparison of the brain to the content-based memory. Multiple locations are searched for the potential match when the input key is entered].

(IP) cores of the CAM based on other memories available on FPGAs. The soft cores are synthesized on the FPGA memories. The synthesized hardware is another type of memory, but it functions like CAM, they are known as emulated CAMs or FPGA-based CAMs. Other researchers have also presented many soft IP core designs to optimize the hardware resources utilization, power consumption, and speed of these FPGA-based CAMs [19].

FPGA-based CAMs are more flexible, i.e., reconfigurable to the need of applications, compared to circuit-level CAMs. Rather than integrating circuit-level CAM in an application, initialization of FPGA-based CAM on the same fabric is preferable for efficient prototyping in searching-based applications on FPGAs [20]. Packet routing in networks, access control systems, pattern recognition, and many other applications have shown improvement on FPGA devices. Thus, an optimal FPGA-based CAM is vital for some major applications to exist. This paper provides the complete picture of FPGA-based CAMs in terms of design improvements over time by different researchers and FPGA vendors. We will briefly discuss the circuit-level CAMs (application-specific integrated circuit (ASIC)-based CAMs) in Section 1.2, and direct the readers to other detailed surveys on ASIC-based CAMs [2,1,21,22]. ASIC-based CAMs are less reconfigurable than FPGA-based CAMs. This paper provides an in-depth survey of the FPGA-based CAM designs that are presented on FPGAs so far, and to the best of our knowledge, this is the first-ever survey of its nature. We have confined the terminologies to the Xilinx-FPGAs-only to make the key concepts easy to understand and not confuse the reader with vendor-specific terminologies. However, we discuss other vendor's FPGAs where it is apparent.

1.1. Focus of this paper

1. This paper covers the FPGA-based CAM/TCAM designs that are presented on different FPGA memories, i.e., block RAM (BRAM), lookup table RAM (LUTRAM), and flip-flops (FFs). We highlight the advantages and disadvantages of every CAM/TCAM design in terms of hardware utilization, power consumption, speed, scalability, updating mechanism, and its other limitations and strengths.
2. This paper provides a taste of CAMs design and development when the available hardware is a high speed, inexpensive, power-efficient, and widely used SRAM memory. Breakthroughs have been made to efficiently emulate CAM using SRAM and optimize it further, which is discussed in this survey. However, we still see research potential in

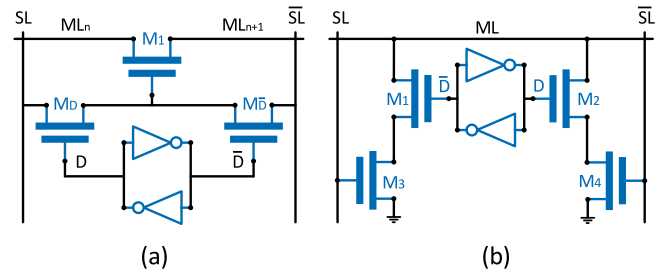


Fig. 2. Conventional binary CAM cells. (a) NAND-cell having 9 transistors. (b) NOR-cell having 10 transistors.

reducing the gap between SRAM and CAM density. This paper serves as a foundation material for the emulation of SRAM memory into CAM/TCAM.

3. This paper puts forward what has been done in the past decade in terms of design improvements, where it stands now, what are the present challenges, what can be done in the future to develop an optimal CAM on FPGAs, and the significant potential applications.

1.2. Conventional CAM

Circuit-level CAM, conventional CAM, transistor-level CAM, typical CAM, traditional CAM; in literature, all of these refer to the ASIC-based CAMs. Binary CAM (BiCAM) and ternary CAM (TCAM) have different hardware cost in terms of the number of transistors per CAM bit. BiCAM supports only '1' and '0', while TCAM supports don't care bit 'X' as well. ASIC-based CAM designs are based on two basic types of CAM cells; NAND-cell and NOR-cell. NAND-cell comprises of 9 transistors while NOR-cell has 10 transistors [23,24]. One NAND-cell compares the stored data and the corresponding search data using three transistors M_1 , M_D , and $M_{\bar{D}}$ bar [25]. Two searchlines and two matchlines in a NAND-cell are represented by SL , $\bar{SL}$, and ML_n , $\bar{ML}_{n+1}$, as shown in Fig. 2(a). One NOR-cell compares the search data and the stored bits using four transistors M_1 to M_4 [26,27]. The searchlines are represented by SL and $\bar{SL}$, while the data lines are D and $\bar{D}$, as shown in Fig. 2(b). The two pairs of transistors M_1 , M_3 , and M_2 , M_4 , creates a pull-down with the ML line.

NAND-cell-based CAM designs have multiple NAND-cells connected in series, which makes them slower than NOR-cell-based CAM designs. On the other hand, the switching probability of matching in a NAND-cell-based CAM design ($1/N$) is lower than in a NOR-cell-based CAM design $((N-1)/N)$ which makes them energy efficient, where N is the number of bits in each CAM word or location or rule [28].

Many studies have covered in detail, the ASIC-based CAMs designed on different modern storage devices such as ReRAM, FeRAM, and others [24,2,29,22,30]. The heart of this paper is not ASIC-based CAMs but to survey all the FPGA-based CAM/TCAM designs on reconfigurable hardware (the FPGA), which are presented by FPGA vendors such as Xilinx and Altera as well as the other memory designers using the available memory resources on FPGA.

2. Background

2.1. Reconfigurable hardware

FPGAs have become increasingly favorable to implement complex reconfigurable systems because of their hardware-like performance and software-like reconfigurability. Its massive hardware parallelism, rapid prototyping capability, and a large number of logic resources make it suitable for ASIC prototyping [31,

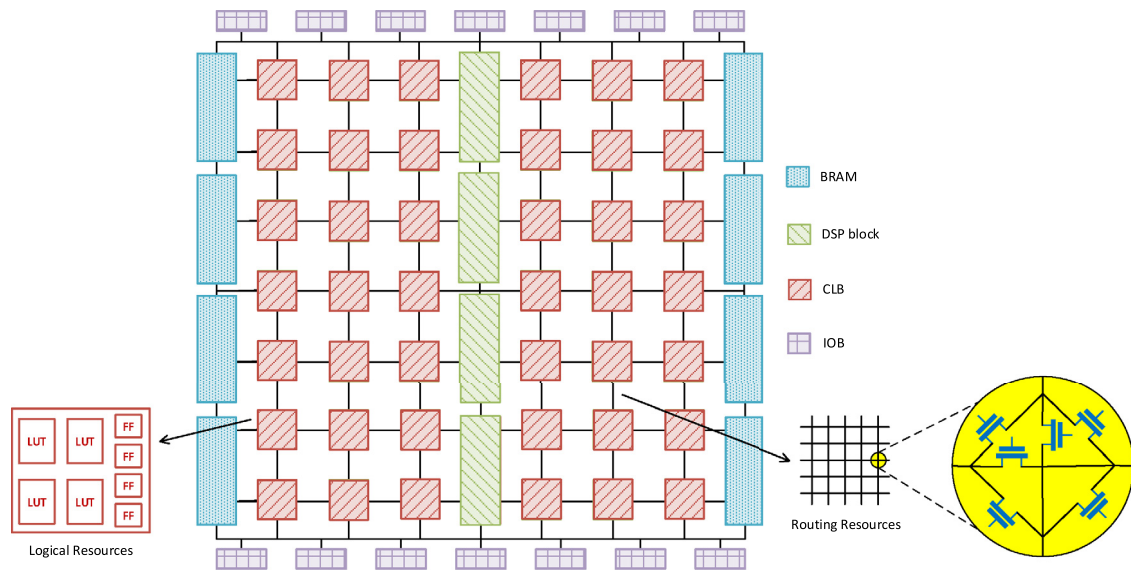


Fig. 3. Modern FPGA architecture. (BRAM: Block RAM, Digital Signal Processing (DSP) block, CLB: Configurable logic block, IOB: Input/Output block).

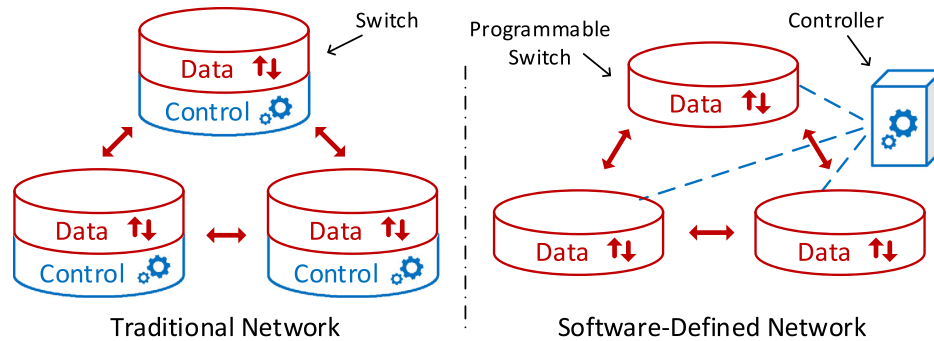


Fig. 4. Conventional or Traditional Network with data and control plane together in the switches vs. Software-Defined Network having control plane shifted to a centralized controller.

17]. Modern FPGAs provide a vast amount of logic and memory resources, and sometimes specific kits, to facilitate the design process of networking applications, e.g., Xilinx provides support for software-defined networks (SDNs).

Fig. 3 shows a generalized FPGA structure. Two major portions are logic resources and routing resources, which are arranged in a matrix form. The logic resources implement logic functions in a digital design while routing resources are used to connect different logic blocks to form the desired digital system. Each switching node consists of six transistors which are controlled using SRAM configuration memory. We confine our discussion to the Xilinx Virtex-6 FPGA device to make the key concepts easy to understand. However, there are many device families from Xilinx, such as Spartan, Virtex, Kintex, Ultrascale, and for each family, there are tens of different devices.

Virtex-6 FPGAs build on a 40 nm process doubled the logic capacity from 360K to 760K logic cells, 207K to 948K FFs, and 8 Mb to 38 Mb BRAM, compared to the successful Virtex-5 FPGA family. The dedicated system-level blocks provide ASIC-like performance by integrating it in the FPGA fabric. The building block of Xilinx FPGA is a configurable logic block (CLB). Each CLB has two slices. Each slice has four lookup tables (LUTs), eight FFs, multiplexers, and carry chain logic. By compromising on four out of eight FFs, the remaining four FFs can be configured as latches.

2.2. Software-defined network

Software-defined network (SDN) uses open protocols (such as OpenFlow) to control the network centrally in an intelligent fashion irrespective of the underlying technology [21]. It can access switches and routers which traditionally are using exclusive and vendor-specific firmware. The key characteristic of SDN is “programmability”, which is aberrant in traditional networks, i.e., the internet. The building blocks of the internet, switches, and routers, are configured manually by network operators. The beauty of SDN architecture is its centralized control, which maintains an overall picture of the network information and states, as shown in Fig. 4. The SDN controllers can configure the routing table (realized using CAM/TCAM) to control the traffic on network building blocks [32]. FPGAs are a favorable platform to implement SDN as it can provide the key requirement of an SDN, i.e., re-configurability and programmability. Thus, it is vital to design an optimal CAM/TCAM for future networks.

2.3. An example application of CAM

CAM is considered to be the cornerstone in networking applications as it implements the routing table, which is required for the classification and forwarding of data packets that arrive at the intermediate nodes [33]. Data in a packet-switched network is divided into packets that are transferred from one node to another using the intermediate nodes called routers. The packet

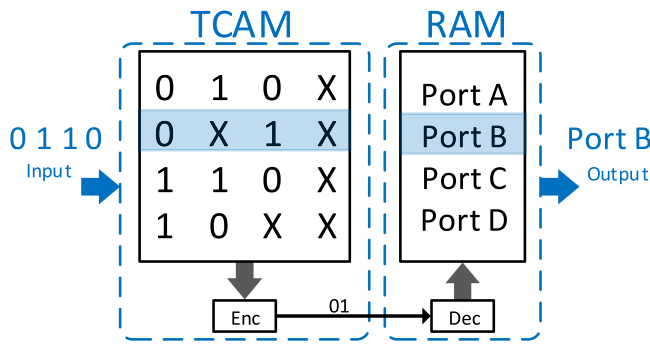


Fig. 5. A Routing table using RAM and TCAM. (Enc.: Encoder, Dec.: Decoder).

consists of a destination address, which is searched by the router in its stored routing table to find the corresponding node at which the packet needs to be forwarded. The routing table consists of destination addresses and the corresponding nodes [34,35]. CAM enables the network router to search the entire routing table in one clock cycle. If the same routing table is implemented using RAM, it takes multiple clock cycles and, in the worst case, equals the number of locations present in the RAM.

Fig. 5 shows a simple implementation of the routing table using RAM and TCAM. It consists of four destination addresses of four bits each stored in TCAM, while their corresponding output ports (connecting to the destination or another router) are stored in RAM. For each output port, there is a range of destination addresses. For instance, Port A is selected if the input address ranges from 0100 to 0101. Port B is associated with 0010, 0011, 0110, and 0111, while Port C is selected if the address is from 1100 to 1101. Similarly, Port D is selected for the input addresses ranging from 1000 to 1011. When an input address of 0110 is provided to the routing table, the word ('0X1X') at the second location of TCAM is matched, which selects the designation output port, Port B, from RAM, as shown in Fig. 5. Hence the packet is now forwarded(routed) to port B. Therefore, Fig. 5 implements a routing table using RAM and TCAM containing the pointers to the corresponding ports for each incoming address, specifically for packet classification and packet forwarding in a network application.

2.4. FPGA vendors CAM supports

Xilinx, Altera, Microsemi, and Lattice Semiconductor are the major FPGA vendors [36]. Altera (now Intel) provided native support for CAM in the form of Embedded System Blocks (ESBs) in older FPGAs (FLEX, Mercury), which can be configured as 32×32 BiCAM but they are no longer supported [37,38]. On the other hand, Xilinx does not provide built-in hardware that can be configured as CAM/TCAM. However, soft IPs are released multiple times, which can be built on its reconfigurable elements (CLBs, RAM blocks, and other elements).

2.5. Memory inside modern FPGAs

As CAMs are implemented on FPGAs using memories, it is imperative to understand the type of memories in FPGAs. Modern FPGAs have three kinds of memory, namely Block RAM, LUT RAM, and Flip-flops.

2.5.1. Block RAM (BRAM)

BRAMs are large blocks of RAM that Xilinx FPGAs provide in the form of 18 Kb and 36 Kb, which can be inferred or declared in many different size configurations.

2.5.2. Lookup table RAM (LUTRAM, also known as distributed memory)

LUTRAMs are 64 or 32 bits column memory with 6 or 5 address bits. One-third of the total LUTs of an FPGA device can be configured as LUTRAMs, while others can only be used for logic functions implementation. The data access in both BRAM and LUTRAM is done sequentially, and only one or (in some cases) two locations can be accessed at a single time. Thus, searching the entire memory requires multiple accesses.

2.5.3. Flip-flop (also known as slice registers)

FFs are one-bit memory elements that can be used to store data as well as can be used as pipeline registers in complex digital design to improve the performance of the system. Multiple FFs are arranged to form a register file where the data is accessed concurrently. We present the available FPGA-based BiCAM and TCAM designs in these three categories with the advantages, disadvantages, and trade-offs of each design. At the end of the discussion, we conclude on which design can suit which scenario for integrating the CAM and TCAM with other complex systems, as in most of the applications, CAM is one of its key elements and not an application in itself.

3. BRAM-based CAMs

Virtex-6 FPGA family consists of 156 to 1064 dual-port BRAMs of 36 Kbits depending on the device and the package size. Each of these BRAMs can be used as two independent 18 Kb blocks of RAM. A clock controls each memory operation (read and write). A memory initialization file (.COE or .MIF) is used to initialize the individual or multiple BRAMs. An increase in width of the required RAM requires connecting multiple BRAMs in parallel with common address lines, while an increase in depth of the needed RAM requires connecting multiple BRAMs in series with common data lines.

3.1. Early patents and simple Brute Force Approach (BFA)

RAM, specifically, static RAM (SRAM), is a widely used memory with low power consumption, low cost, and high storage density compared to CAM- and TCAM-cell [39] as discussed in Section 1.2. Furthermore, RAM can be used to implement a TCAM. The first simple approach to make RAM functions like CAM uses a Brute Force Approach (BFA) which presents the idea to use the address of RAM as the search key of CAM, while data of RAM stores the information about the address of CAM [40–43]. Each bit in the RAM dictates the existence of the CAM word. The data bit position in RAM corresponds to the address location of CAM. Fig. 6 shows an example of 8×2 CAM, which uses 4×8 RAM with an 8:3 priority encoder. Location 5 in CAM stores "1 0" which is mapped to the bit position 5 at location 2 in RAM. When $(10)_2$ is provided to the 4×8 RAM as input, the eight-bit data "0 0 1 0 0 0 0" is taken as output. The output of RAM is converted to 3 bit address by the priority encoder, which is 5 in this case where the search key "1 0" has been found.

The drawback of BFA is the inefficient usage of RAM. The requirement of memory to emulate CAM becomes double with an increase of a single bit in the CAM width, and the growth in the required memory is exponential. Fig. 7 shows the emulation of different size CAMs from RAM using BFA, which indicates that the required RAM doubles for an increase of one bit in the width of CAM. Even a small size of 64×36 TCAM requires a large amount of RAM (68 Gbits), which is not possible inside FPGAs. This size of priority encoder also increases with the increase in depth of CAM.

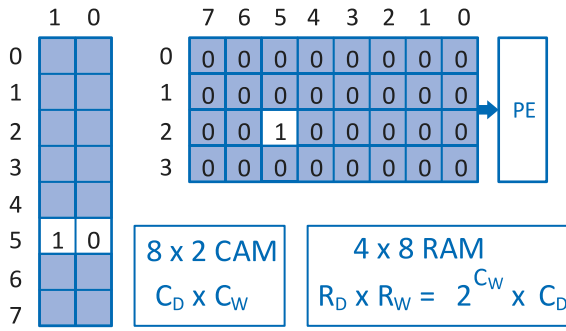


Fig. 6. Emulation of 8×2 CAM from 4×8 RAM using Brute Force Method. (C_D : Depth of CAM, C_W : Width of CAM, R_D : Depth of RAM, R_W : Width of RAM).

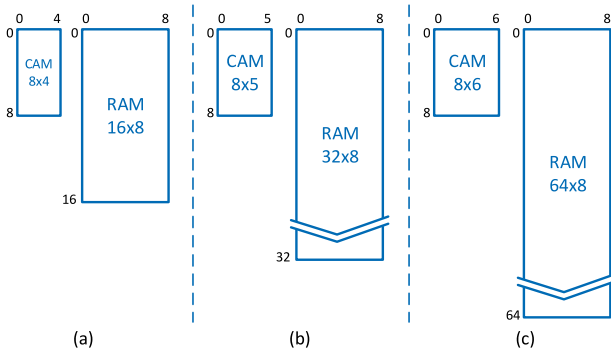


Fig. 7. Increase in hardware requirement using Brute Force Method. (a) 8×4 CAM needs 16×8 RAM. (b) 8×5 CAM needs 32×8 RAM. (c) 8×6 CAM needs 64×8 RAM. An increase of one-bit in the width of CAM doubles the requirement of RAM.

Somasandaram et al. presented a similar architecture in a patent [44], but instead of using a priority encoder, it used an additional memory to find the index (address) of the matched pattern. In every location of RAM, it takes extra bits to store the Last Index (LI), which translates the number of words stored in the previous locations. The increase in hardware resource requirement is still exponential, and this technique is also unable to implement a size of 64×36 CAM inside an FPGA. Besides the Somasandaram et al. [44], some patents from FPGA vendors (Xilinx and Altera) also use the BFA to implement CAM functionality from RAM resources [42,43].

3.2. Partitioning of CAM

A significant problem in the BFA is the large address space of required RAM. The only solution is partitioning the address space by dividing the width of CAM/TCAM. The first attempt of such partitioning was HP-TCAM and was successful to enable the emulation of TCAM using FPGA resources. Ullah et al. proposed the solution to the exponential growth in the resource requirement of RAM-based CAM in the form of a hybrid partitioned TCAM known as HP-TCAM [45]. Separate blocks of RAM (BRAM in Xilinx FPGA) store the information about the presence of TCAM word and its address. A 512×36 TCAM is implemented using 56 BRAMs of 36 Kb. The TCAM is divided into vertical and horizontal partitions, known as hybrid partitioning of the whole memory. Address position tables (APT, to generate the address) and bit position tables (BPT, to validate the presence of input search key), are stored in different BRAMs. The incoming search key is divided into n w -bit sub-words. Each sub-word is fed to its corresponding BPT to validate the presence of the TCAM sub-word. The last Index (LI) from BPT generates an address (APTA) of a table (APT)

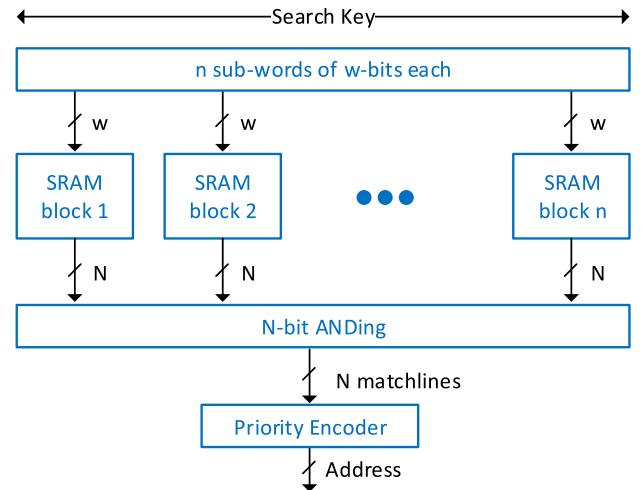


Fig. 8. UE-TCAM Architecture. One SRAM block is a single or a collection of BRAMs in Xilinx FPGAs.

where the information about the location is found. The data from n tables (APTs) are ANDed to form N matchlines, where N is the number of total words in TCAM. Finally, a priority encoder generates the potential match address from N matchlines. HP-TCAM inefficiently utilizes memory resources to store the presence and address of stored data in separate memory blocks. TCAM data requires to be in ascending or descending order; otherwise, HP-TCAM completely fails to provide the correct address. Insertion or removal of TCAM words into HPTCAM is not discussed in this work, which is a separate hardware overhead proposed after a few years. The search latency of HP-TCAM is five clock cycles.

HP-TCAM is now obsolete and extremely complex compared to the simple and efficient partitioning in UE-TCAM, which you will agree upon in the next few sections.

3.3. Further partitioning

With a few small changes, Z-TCAM and E-TCAM [46,47] reduced the latency from 5 to 4 and 3 clock cycles, respectively, but the memory usage is almost similar to HP-TCAM. Instead of APTs, original address tables (OAT) were introduced, and the intermediate signals were eliminated. Zerbini et al. also presented the idea of using cascaded TCAM blocks to perform packet classification [34].

3.4. UE-TCAM

An advancement in the FPGA-based TCAM is the ultra-efficient (UE)-TCAM [48], which reduced the memory utilization to half of the HP-TCAM. A simple architecture is presented, which inherits the validation memory from HP-TCAM in a different way. The matchline vectors are stored as they are in the BRAM memory. When a search key is provided to the UE-TCAM, it is divided into n w -bit sub-words which are fed to the corresponding SRAM blocks, as shown in Fig. 8. Matchline vectors from n SRAM blocks are ANDed together to form the resultant matchline vector which represents the whole search key. Finally, a priority encoder generates an address from the resultant matchline vector.

Fig. 9 highlights the key concept of partitioning in reducing the hardware utilization from BFA to UE-TCAM. A 512×36 CAM is presented using both techniques; BFA and UE-TCAM. The RAM resources required by BFA are 68 Gb, as shown in Fig. 9(a), while UE-TCAM requires only 1 Mb with a slight overhead of logic resources in the form of ANDing operation, as shown in Fig. 9(b).

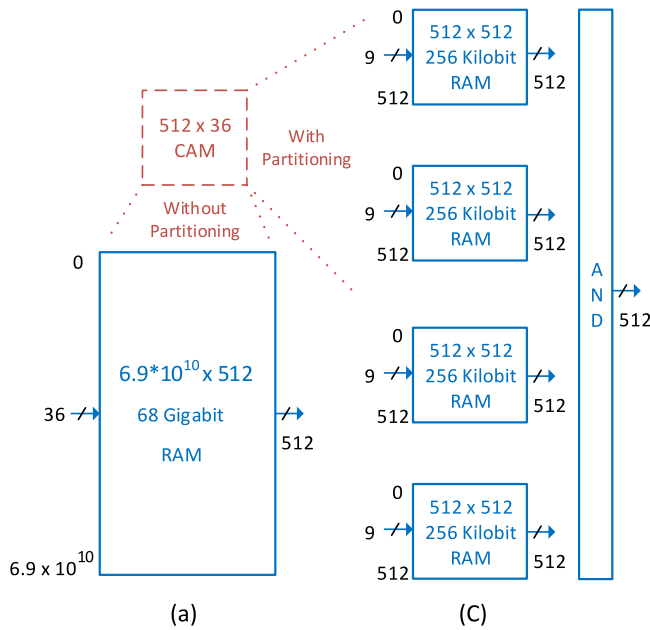


Fig. 9. An example (512×36) emulation of CAM using: (a) Brute Force Approach (BFA), which does not include any partitioning. (b) UE-TCAM, which involves partitioning of the address space.

3.5. Xilinx CAMs

Xilinx is a leading vendor in the FPGA industry and shares a prominent number in the FPGA market. FPGA domain is a comprehensive developmental tool that is reconfigurable according to the need of an application. A hard-core IP for CAM takes part in the FPGA fabric, which may not be useful for other applications. To maintain the salient characteristic (reconfigurability) of FPGAs, a hard-core IP is not included in modern devices. However, to support CAM-based applications on FPGAs, Xilinx presented a soft IP core in 2011 and recently released a soft IP for software-defined networks in 2017.

Xilinx presented two types of CAMs in its application note in 2011 [49]; one using BRAM (BRAM), and others using 16 bit shift registers known as SRL16E in Xilinx terminology. The latter is discussed in Section 4.1. The BRAM-based CAM architecture does not support the storing of do not care bits and is only a BiCAM. It used 32 BRAMs for a 256×32 CAM. It seems to be very rarely used in any practical application and Xilinx has not updated the application note since its first release, XAPP1151 (v1.0), in March 2011.

Looking into the recent development, a large number of publications, need for a softcore CAM, and further optimization in FPGA-based CAMs, Xilinx released a soft IP core [50] for TCAM, which is implemented using BRAM and LUTRAMs. Although it reduces the usage of BRAMs, an increase in slice utilization degrades the overall hardware utilization.

3.6. Memory and power efficient TCAM

A hybrid architecture containing RAM, as well as conventional CAM, is presented in the patent [51], which inherits the instinctive drawbacks of conventional CAMs. The CAM entries are divided into specific groups based on distinguishing bits such that only one entry from each group can match with the search input. Dividing the TCAM entries into groups requires pre-processing, and the optimal number of groups with distinguishing bits is not known. However, using a simple heuristic approach Yu et al. [52]

uses encoding to compress the search key and partitions the ruleset. Results have shown higher memory efficiency in packet classification with large rule sets, i.e., 10K to 100K rules.

3.7. Hierarchical CAM

To reduce the power consumption, a hierarchical design of RAM blocks is presented in [53], where the blocks are activated based on their priority. The search key is rippled from block to block until the match is successful. If the search key is found in the earlier RAM blocks, the latter are skipped in the search operation, which reduces the power consumption, but the worst-case power consumption remains high. The latency of TCAM is compromised to 7 and 9 clock cycles by dividing the architecture into 7 and 9 hierarchical stages of BRAMs and distributed RAMs, respectively. The low priority blocks are skipped if the search key is found in high priority blocks, which results in saving the power consumption of the whole design. In BRAM-based implementation and LUTRAM based implementation, the reduction in power consumption is 11% and 9.7%, respectively, for a 504×180 TCAM.

3.8. REST

A resource-efficient SRAM-based TCAM (REST) architecture is presented in [54] to decrease the hardware utilization of FPGA-based TCAM. Speed is compromised to achieve a tremendously low hardware utilization. A TCAM of size 72×28 is implemented using only one BRAM of 36 Kb, which required 54 BRAMs of 36 Kb in the case of HP-TCAM design. REST virtually increases the address space of the mapping RAM blocks by accessing the same RAM block multiple times for a single search operation. The latency of REST is 5 clock cycles for size of 72×28 TCAM on Xilinx Kintex-7 FPGA.

3.9. Energy efficient (EE) - TCAM

An energy-efficient TCAM (EE-TCAM) is presented in [55], which divides the TCAM table into sub-tables based on classifier bits. Each sub-table is mapped to a separate RAM sub-block. This is similar to the memory and power-efficient architecture presented in patent [51], which is discussed in Section 3.6, where these classifier bits were named as distinguishing bits. Dividing the CAM table into sub-tables based on individual bits is a tedious job and does not guarantee an optimal solution for sub-tables. This becomes so costly in terms of time as well as hardware utilization when the TCAM table is totally random and frequently updatable. However, three times power reduction is achieved on Xilinx Virtex-6 FPGA compared to UE-TCAM [48].

3.10. Scalable BRAM-based TCAM

Scalable TCAM implemented the largest size TCAM (4048×180), which also uses the BFA to emulate the TCAM functionality [56,57]. A high-end FPGA device Xilinx Virtex-7 with the most extensive hardware resources is used. The energy consumption of scalable TCAM is high compared to the other BRAM-based TCAM architectures.

3.11. Multi-pumping multi-ported TCAM

A similar approach as REST [54] is presented in multi-pumping multi-ported TCAM [58] with a slightly different design to save memory resources. It accesses the BRAM multiple times for one search operation, which improves hardware utilization. Multi-pumping means to pump separate addresses in one system clock cycle. Two things are combined in this design: multi-pumping

Table 1
Summary of BRAM-based CAMs.

Architecture	Size of CAM	Important features
HP-TCAM [45]	512 × 36	+ Implemented on FPGA for the first time – No discussion about updating the design – Data needs to be in ascending/descending order – Preprocessing of data is required
Z-TCAM [46], E-TCAM [47]	512 × 36	+ Search latency reduced – No discussion about updating the design – Data needs to be in ascending/descending order – Preprocessing of data is required
UE-TCAM [48]	512 × 36	+ Hardware utilization improved + Search latency reduced + No need of data to be in ascending/descending order – No discussion about updating the design
REST [48]	512 × 36	+ Hardware utilization improved – Search latency degraded – Speed degraded
EE-TCAM [55]	512 × 36	+ Reduced Power consumption – Search latency degraded
Scalable TCAM [57]	1024 × 150	+ Update modules presented – Architecture similar to [48] presented with complex terminologies
Hierarchical TCAM [53]	–	+ Hardware utilization improved – Low latency
Multipumping TCAM [58]	512 × 36	+ Hardware utilization improved – Low latency – Speed of 133 MHz
Xilinx CAM (2011) [49] (BiCAM)	1024 × 32	– No support for ternary CAM using BRAMs – Low speed of 80 MHz on Xilinx Virtex-5 FPGA
Xilinx TCAM for SDNet (2017) [50]	512 × 40	– 13 times more slices than UE-TCAM used
Others		
Index Generator [44]	512 × 36	– Not Implementable on FPGA
Power Efficient Lookup [51]	512 × 36	+ Hardware Utilization reduced compared to [44] – Not Implementable on FPGA – Requires conventional CAM

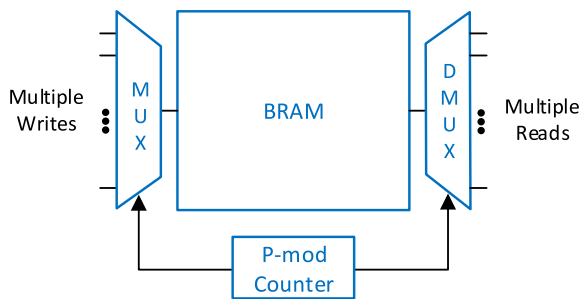


Fig. 10. Multi-pumping multi-ported TCAM. (MUX and DMUX are the multiplexer and demultiplexer. BRAM is a RAM block in Xilinx FPGAs. P-mod Counter is set to the number of accesses to BRAM for one search operation which determines the latency of the whole design).

and multi-ported of memory blocks inside FPGAs. Exploiting the multi-ported RAM blocks of FPGA are advantageous to reduce the hardware resources, but multi-pumping, similar to REST, is saving hardware resources at the cost of latency. Fig. 10 shows the design of multi-pumping multi-ported TCAM in which the BRAM is accessed multiple times using a p mod counter, where p is the number of accesses in one system clock cycle.

3.12. Combining all block RAM based memory

Table 1 summarizes the key features of BRAM-based CAM architectures that are presented/implemented for/on FPGAs. FPGA-based TCAMs are still not as scalable as ASIC-based CAMs because of the limited resources on the FPGA devices. R8A20410BG is a

commercial ASIC-based TCAM of size 40 Mb from Renesas [59,60, 8]. In contrast, FPGA-based CAM/TCAM are limited to the size of 1024*180 bits except [52] having 15 Mb TCAM using a heuristic approach. The reason for this restraint is the minimal resources on FPGAs, although they are increasing day by day and almost double in every new release. Secondly, the structure of the BRAM in modern FPGAs limits the mapping of TCAM data in a certain way that RAM/TCAM bit ratio is high, which can be seen in the next sections that LUTRAM-based TCAMs can achieve a lower RAM/TCAM bit ratio. If by some means, the ratio has been brought closer to 1, it will be a breakthrough and maybe a stop to the research in emulation of CAM/TCAM on FPGAs.

4. Distributed RAM-based CAM/TCAM

Logic gates are available in FPGAs in the form of LUTs, which are also known as function generators because an n -input LUT can implement any n variable logic function. A 3-input LUT is an 8 bit column memory where the 3 bit input serves as the address to access each 1 bit data in 8 locations, and the 1 bit data acts as the output of the given function. For instance, a 3-input AND gate is implemented using 3-input LUT, as shown in Fig. 11.

BRAMs are often reserved for the exact match operation in SDN, in which case, the BRAM resources can be insufficient to deploy a TCAM. Distributed memory is the second memory resource (after BRAM) on an FPGA. It utilizes the slice resources to realize any size of SRAM block by combining the LUTRAMs available in sliceM.

4.1. Xilinx application note (SRL16E based TCAM)

Xilinx presented two architectures in their application note in 2011; BRAM-based BiCAM discussed in Section 3.5, and the

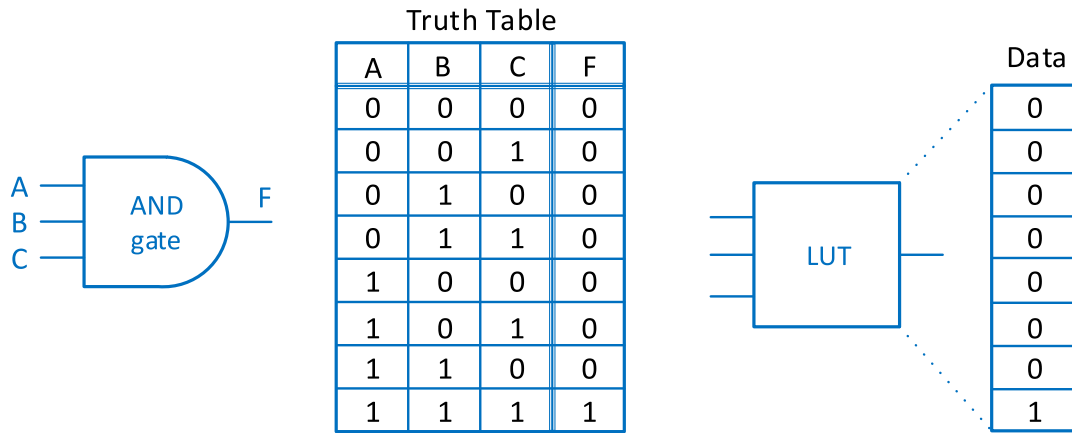


Fig. 11. Lookup table as function generator. (LUT: lookup table; A, B, and C are the 3 inputs, while F is the output of AND gate as well as LUT).

SRL16E which we will discuss now. The SRL16E is a 16 bit shift register, which is also known as a four-input LUTRAM. SRL16E primitive is used as a column memory of 16 locations, and two ternary bits are implemented using one SRL16E block. It is efficient in hardware utilization than the block RAM implementation in terms of SRAM cells per CAM bit, but due to the shallow RAM blocks, the routing complexity degrades its performance. Extra hardware is used to mask the do not care bit in the form of comparators which is an overhead compared to the BRAM-based implementation. The update process needs 16 clock cycles to overwrite all of the corresponding LUTRAMs which is less than the 512 clock cycles of BRAM-based implementation.

4.2. Scalable LUTRAM-based implementation

A simple mapping architecture similar to the BFA is presented in [57], which reduced the number of LUTRAMs used by Xilinx implementation. The difference from the Xilinx SRL16E based implementation is that it used 64 bit LUTRAMs of Xilinx Virtex-7 FPGA and implemented a sufficiently large size. The speed (clock rate) of the design depends on the arrangement of the hardware when such complex systems are designed.

In all of the discussed designs so far, the update operation halts the search operation, which affects the high-speed searching capability of CAM/TCAM. There are some proposals such as the DURE TCAM discussed in 4.3 that improve the update operation without affecting the search operation.

4.3. DURE TCAM

DURE TCAM [61] is a TCAM purely based on LUTRAMs that focused on improving the update efficiency and solves the problem of update locking. During the update process, the FPGA-based TCAM based on BRAM, as well as LUTRAM, are not available for the search operation, which is known as update locking. For the 64 clock cycles in the case of LUTRAM and 512 clock cycles in the case of BRAM, the TCAM suspends its search operation. DURE dynamically updates the TCAM as there are separate pins for writing and reading of the SRAM memory. Only that location cannot be searched for the possible match, which is busy in the operation of update. Efficiency in terms of hardware resources, latency, and power consumption is achieved by DURE TCAM compared to the other FPGA-based CAMs. It used 4 LUTRAMs to implement 1×18 TCAM, so the SRAM per TCAM bit ratio is 14.22, which is higher than the Xilinx SRL16E implementation where SRAM/TCAM bit ratio is 8. It is because of the fact that, Xilinx used shallow SRAM blocks, which we have already stated that shallower SRAM blocks

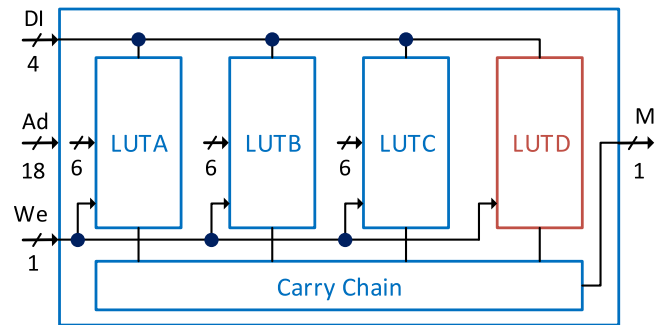


Fig. 12. Building block of DURE implementing 1×18 TCAM using one slice. LUTA, LUTB, and LUTC are used to store each of six bit chunks of 18 bits of TCAM. LUTD is not used to store the data of TCAM due to the hard-wired structure of a slice in Virtex-6 FPGA. (DI: Data In, Ad: Address/Search key, We: Write enable, M: Match/Mismatch).

will reduce the hardware utilization of the TCAM design, but still, the performance of DURE is better than Xilinx SRL16E TCAM.

Fig. 12 shows a simplified building block of DURE, which implements 1×18 TCAM using one slice of Xilinx Virtex-6 FPGA. The width of the TCAM is divided into 6 bit chunks to map it to LUTA, LUTB, and LUTC to realize a total of 18 bits with one slice. Two slices are required to implement 1×36 TCAM. The beauty of DURE is that it reduced the hardware cost by using the carry chain circuit for comparison.

4.4. PR-TCAM

Pedro et al. presented a TCAM architecture based on LUTs, however, it is different from other TCAM designs discussed from Sections 4.1 to 4.3. Xilinx FPGAs have two types of slices which consist of LUTs. SliceM slice consists of LUTs, which can be used as memory (known as LUTRAMs) as well as for logic implementation. SliceL slice consists of LUTs, which can only be used for logic implementation. All of the previously discussed LUT-based TCAMs use LUTRAMs as SRAM blocks, while PR-TCAM uses LUTs of SliceL. The hardware utilization is reduced to half by using LUTs from SliceL instead of LUTs from SLICEM [62]. There is a major problem with SliceL LUTs, that is, they cannot be updated similar to other SRAM blocks. However, modern FPGAs provide the facility of updating LUTs of SliceL by partial reconfiguration, which is exploited by PR-TCAM [63,64]. The update time is in the range of milliseconds, which is not feasible for dynamically updatable networking applications.

Table 2
Distributed RAM-based CAMs.

Architecture	Size of CAM	Important features
Xilinx CAM 2017 [44]	1024 × 64	– Inefficient use of LUTRAMs – A low speed of 70 MHz on Virtex-5 FPGA
Scalable TCAM [57]	4096 × 150	+ Implemented a large size on FPGA – Complex terminologies are used + Update module presented
Hierarchical TCAM [53]	504 × 180	+/- Low power for specific data set – Low latency – Speed of 109 MHz on Virtex-6 FPGA
Zi-CAM [65]	512 × 36	+/- Low power for specific data set + Bypassing scheme is presented – Achieves very low speed – Does not support ternary bits
PR-TCAM [62]	512 × 40	+ Partial reconfiguration introduced + Make use of logic LUTs in SLICEL – Update process is slow
D-TCAM [66]	512 × 36	+ High throughput – No module for update
DURE [61]	512 × 36	+ Dynamically updatable – Low throughput than D-TCAM

Xilinx FPGAs have reconfigurable slices that consist of LUTs and FFs, as we have seen in Section 2.1. Both of them are memory elements that can both be used to emulate a single TCAM.

4.5. D-TCAM

D-TCAM exploits the paired nature of LUTs and FFs in Xilinx FPGAs to present a high throughput TCAM [66]. As we discussed in Section 2.1, slices are the major components in an FPGA that are assigned by the synthesizer to a digital design. The hardware utilization is in terms of slices whenever the logic elements such as slice registers (SRs) or lookup tables. FF-based TCAM architectures utilize only FFs to realize TCAM on FPGA, but the paired lookup tables are left unused. Similarly, LUTRAM-based TCAM architectures utilize only LUTs (distributed memory) to emulate memory on FPGA but the paired FFs are left unused. D-TCAM is similar to the LUTRAM-based TCAM architectures to store the TCAM data, but the unused FFs act as pipeline registers, as shown in Fig. 13. Pipelining of the LUTRAM-based TCAM improved the speed of the presented architecture without any additional hardware resources. FracTCAM [67] utilized the built-in FFs of the slices for improving the speed of the LUTRAM-based TCAM at the slice level. It is somehow similar to the D-TCAM architecture.

4.6. Updating mechanisms

Update means writing of new data into TCAM. Update latency refers to the number of clock cycles taken by a design to overwrite a location of CAM/TCAM [32]. Without the update module, CAM is incomplete. FPGA-based CAMs using BFA [44,51] were not implementable on FPGA resources. HP-TCAM reduced the memory requirement but has not discussed the update mechanism of the presented design. Farkhanda et al. in 2017 came up with a design [68] to update the HP-TCAM in a known number of clock cycles.

In BRAM-based TCAM implementation, the update latency is very high, which degrades the performance of the whole system and makes the FPGA-based TCAM infeasible for practical applications. Therefore, a few techniques [68,69,57] are proposed to improve the latency of BRAM-based TCAMs.

Due to the high-depth RAM structure of BRAM, it takes a large number of cycles to be updated. On the other side, the LUTRAM address space is lower than BRAM; it takes a lesser number of cycles to be updated. So the update latency of BRAM-based CAMs

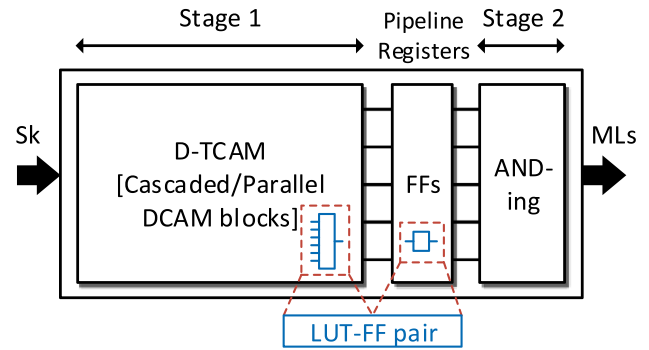


Fig. 13. LUT-FF pair in a slice of an FPGA [66]. Two stages (storing memory and ANDing operation) are separated by flip-flops which act as pipeline registers. (Sk: Search key, MLs: Matchlines).

is higher than the LUTRAM-based CAMs. In Xilinx FPGAs, a BRAM is updated at least in 512 clock cycles, while a LUTRAM takes only 64 clock cycles to be updated completely. However, methods for reducing the update latency of BRAM-based TCAMs is a potential research direction on FPGA-based TCAMs.

4.7. Summary of the distributed-RAM FPGA based TCAMs

Table 2 summarizes the LUTRAM-based TCAMs. LUTRAMs are shallow RAM blocks compared to BRAMs, which makes it possible to reduce the RAM/TCAM ratio from 64 to 14, with the tradeoff of large bit-wise ANDing circuitry. So far, DURE and D-TCAM are the most feasible implementations of TCAM using distributed RAM, while PR-TCAM can scale further as it can cover both sliceM and sliceL to use LUTs.

5. Flip-flop-based CAMs

5.1. LH-CAM

A BiCAM is presented in [70], which used only one flip-flop and one-bit comparator to implement a BiCAM cell, as shown in Fig. 14(a). Each cell is provided with a comparator that compares the incoming search word bit-by-bit with each LH-CAM cell in parallel. The number of flip-flops required for LH-CAM implementation is simply a multiplication of the number of rows in the

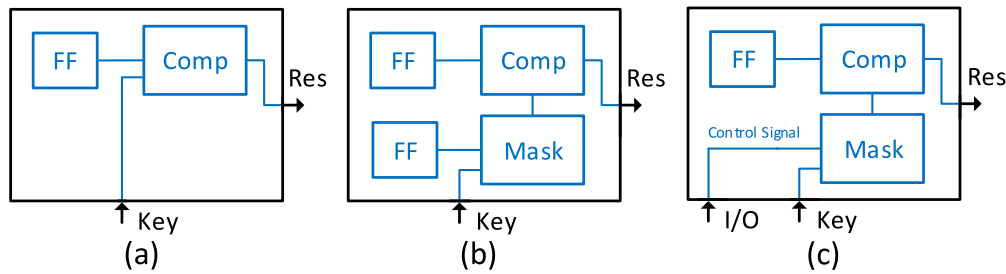


Fig. 14. A building block of: (a) LH-CAM. (b) G-AETCAM. (c) RE-TCAM. (FF: flip-flop, Comp: one-bit comparator, Mask: an OR gate, I/O: Input/Output pin of FPGA, Key: one bit of input search key, Res: Result of comparison which can be a match or mismatch).

Table 3

Flip-flop (FF)-based CAMs.

Architecture	Size of CAM	Important features
LH-CAM [70]	64×36	+ Improved speed from BRAM-based CAMs – Not scalable due to routing complexity
G-AETCAM [71]	64×36	+ First FF-based TCAM + Improved speed from BRAM-based designs – Not scalable
RE-TCAM [72]	64×36	+ Reduced hardware resources – Masking of input data is required – Larger number of I/O pins are required

BiCAM and the number of bits in each row. A BiCAM of 64×36 is implemented on Xilinx Virtex-6 using only 2304 flip-flops, which are also known as slice registers (SRs). This design has a low hardware cost than BRAM-based and LUT-based designs but is suitable only for small sizes because of its routing complexity. LH-CAM suffers from the issue of scalability, which is essential in networking applications as wider CAMs are needed to store the IP addresses. However, it can be used in specific applications where the size of the required CAM is small.

5.2. G-AETCAM

G-AETCAM is a ternary CAM version of LH-CAM, which used two flip-flops to implement one TCAM bit [71], as shown in Fig. 14(b). The binary CAM is converted to a ternary CAM by adding a second flip-flop per Bi-CAM cell of LH-CAM. One flip-flop stores the memory information that can be logic 0 or logic 1, while the second flip-flop is used to mask the stored information to a do not care bit. The two flip-flops are named as a memory element and masking element, storing the actual binary bit and masking bit, respectively [20]. It used 4608 flip-flops (slice registers) on Xilinx Virtex-6 FPGA to implement a TCAM of size 64×36 . The speed and hardware utilization of flip-flop based BiCAM and TCAM is better than the BRAM and LUTRAM-based CAMs, but it suffers from the routing complexity which affects the scalability of such CAM designs. However, it can be preferred over the other TCAM architectures in which the size of required TCAM is small. In gate-based area-efficient (G-AE) TCAM, each cell is formed by two flip-flops, a comparator, and a NOR gate to mask the stored bit. The initialized FFs show a reduction in power consumption and slightly improve the speed for the first search operation [31]. G-AETCAM uses two FFs per TCAM bit, which limits its scalability to realize higher-order TCAMs.

5.3. RE-TCAM

A TCAM design, RE-TCAM, is presented by Mahmood et al. to reduce the number of FFs per TCAM bit of the G-AETCAM. RE-TCAM uses one FF and a control signal instead of two FF per TCAM bit [72], as shown in Fig. 14(c). The masking element of G-AETCAM is eliminated using an input/output (I/O) pin of the

FPGA as a control signal, which is not feasible for a considerable size of TCAM. For example, 64×36 TCAM requires 2304 I/O pins to realize the control signals of RE-TCAM. Global masking justifies its feasibility on FPGA to reduce the number of required I/O pins in which multiple cells share a single control signal. For example, if the first bit of all stored words in 64×36 TCAM is an X-bit, only 1 I/O pin is required as a control signal for all of the 64 locations instead of 64 I/Os. Global masking enables RE-TCAM to be implemented on FPGA but limits its generalization, and only applies in particular applications.

5.4. Summary of the flip-flop based TCAMs

FF-based CAMs are less scalable compared to BRAM-based and LUTRAM-based CAMs. For small size CAMs, e.g., 64×36 , it proves to be more efficient in terms of throughput (speed), power consumption, and hardware resources. G-AETCAM is so far the best design compared to other designs. Each slice of the Xilinx FPGA has eight FFs that can be utilized to design FF-based TCAM and the carry chain can be used for comparison, which is previously done using LUTs in G-AETCAM. Future TCAM designs can come up to decrease the number of required FFs per TCAM cell, which can make it more scalable, and it is a clear research direction. (See Table 3.)

5.5. Some other types of CAMs

5.5.1. Hash-based CAMs

Hash-based CAMs provide search results according to the hash function result using a portion of the input search key. However, a perfect hash function does not exist leading to a significant problem with hash-based CAMs [23,73] in the form of its non-deterministic search latency. Hash-based CAM suffers from collision where multiple keys match to a single entry. BiCAM seems to work somehow efficiently using Cuckoo hashing with a slight hardware overhead but the presence of wild cards (do not care bits) makes it perform worst. The probability of collision depends on the hash function, which is not possible to be perfect, especially in the case of random and frequently updatable data.

Table 4

Results of different BRAM-based CAM/TCAM architectures on FPGAs. (Gbps: Gigabits per second, cc: clock cycles, fJ/bit/search: femtoJoule per bit per search, BRAM: Block RAM, LUTRAM: Lookup table RAM, FF: Flip-flop).

CAM/TCAM design	CAM/TCAM size	FPGA device	Hardware utilization	Throughput (Gb/s)	Latency (in cc)	Energy consumption (fJ/bit/search)
BRAM-based CAM/TCAM architectures						
HP-TCAM [45]	512 × 36	Virtex-6	56 (36k) BRAMs	4.25	5	102.2
UE-TCAM [48]	512 × 36	Virtex-6	32 (36k) BRAMs	7.26	2	42.3
REST [54]	72 × 28	Kintex-7	1 (36) BRAMs 77 Slices	0.98	5	798
EE-TCAM [55]	512 × 36	Virtex-6	32 (36) BRAMs	11.3	5	15.03
Scalable TCAM [57]	1024 × 150	Virtex-7	272 (36) BRAMs 12138 Slices	20.8	—	180
Hierarchical TCAM [53]	504 × 180	Virtex-6	140 (36) BRAMs	23.38	7	280.86
Multipumping TCAM [58]	512 × 32	Virtex-6	16 (36) BRAMs	7.41	—	—
Xilinx 2011 TCAM [49]	256 × 32	Virtex-5	104 Slices	5.2	1	68
Xilinx 2017 TCAM [50]	512 × 128	Virtex-7	3 (36K) BRAM 12011 Slices	15.36	8	220

Table 5

Results of different LUTRAM-based CAM/TCAM architectures on FPGAs. (Gbps: Gigabits per second, cc: clock cycles, fJ/bit/search: femtoJoule per bit per search, BRAM: Block RAM, LUTRAM: Lookup table RAM, FF: Flip-flop).

CAM/TCAM design	CAM/TCAM size	FPGA device	Hardware utilization	Throughput (Gb/s)	Latency (in cc)	Energy consumption (fJ/bit/search)
LUTRAM-based CAM/TCAM architectures						
Hierarchical TCAM [53]	504 × 180	Virtex-6	10067 Slices	19.26	9	263
Zi-CAM [65] (BiCAM)	512 × 36	Virtex-6	1 (36K) BRAM 814 Slices	1.33	3	50.6
PR-TCAM [62]	512 × 40	Artix-7	894 Slices	3.20	—	—
Xilinx 2017 TCAM [50]	512 × 128	Virtex-7	3 (36K) BRAM 12011 Slices	15.36	8	220
Scalable TCAM [57]	4096 × 150	Virtex-7	20526 Slices	20.9	6	180
D-TCAM [66]	512 × 36	Virtex-6	968 Slices	16.6	—	—
DURE TCAM [61]	512 × 36	Virtex-6	1668 Slices	12.06	1	28

Table 6

Results of different FF-based CAM/TCAM architectures on FPGAs. (Gbps: Gigabits per second, cc: clock cycles, fJ/bit/search: femtoJoule per bit per search, BRAM: Block RAM, LUTRAM: Lookup table RAM, FF: Flip-flop).

CAM/TCAM design	CAM/TCAM size	FPGA device	Hardware utilization	Throughput (Gb/s)	Latency (in cc)	Energy consumption (fJ/bit/search)
FF-based CAM/TCAM architectures						
LH-CAM [70] (BiCAM)	64 × 36	Virtex-6	315	11.9	2	62.5
G-AETCAM [71]	64 × 36	Virtex-6	774	10.97	3	156.25
RE-TCAM [71]	64 × 36	Virtex-6	629	12.23	2	121.5

5.5.2. Soft errors in CAMs

Memories in modern electronic circuits are susceptible to errors where a few bits are altered, which can be extremely crucial in time-critical systems [74,64]. FPGA-based CAMs are designed on top of the SRAM. The bit error rate is 10^{-9} errors per year, which is very low. However, it can be a big concern in network routers, which are deployed in reliable and zero-error communication systems.

6. Combining them ALL

We have covered all the three types of memories in FPGAs that how they are utilized to implement CAM/TCAM, i.e., Section 3 about block RAM (BRAM), Section 4 about distributed RAM or LUTRAM, and Section 5 about flip-flops (FFs). Since the intrinsic nature of these three memories is different at the hardware level, a direct comparison in terms of performance parameters may not be fair, which is why we have presented it differently. Rather than comparing in terms of performance parameters, they are useful in different application scenarios as will be discussed in Section 8. BRAM-based TCAM requires pre-processing for mapping data, and has a large SRAM/TCAM bit ratio of 16, while LUTRAM-based

CAM/TCAMs has an SRAM/TCAM bit ratio of 14 [61,50]. Since individual FFs are accessible in FPGAs, the hardware cost of FF-based CAM/TCAM is lower than the BRAM-based and LUTRAM-based CAM/TCAM.

7. Performance metrics and results

HP-TCAM and Xilinx in 2012 implemented a TCAM on Xilinx Virtex-6 FPGA to do the performance evaluation. Different FPGA devices have different performances. For instance, Xilinx Virtex series are high-performance devices, Artix is a low power, while Spartan is slightly compromised on both. To develop a new design for CAM/TCAM on FPGA, it is essential to compare the performance with state-of-the-art works. In literature, most of the works are implemented on Xilinx Virtex-6 and Virtex-7 FPGA. Therefore, we take Xilinx Virtex-6 as a base architecture to explain the emulation of CAM/TCAM. However, other FPGA devices are one or the other way similar, and the implementation results are slightly different.

Xilinx Virtex-6 is a device from a high-speed FPGA family consisting of two types of slices; sliceM and sliceL [75]. SliceM is used for memory as well as logic, while sliceL can only be used for logic

Table 7
Important Hardware Resources available on Xilinx Virtex-6 Device XC6VLX760.

Slices		BRAMs	Max User I/Os	DSP slices
SliceL	SliceM	(36 Kb)		
85440	33120	720	1200	864

implementation. The LUTs available in sliceM are also known as LUTRAMs, as these are small column RAMs of 64 locations. Every FPGA device has 25 to 30% sliceM while the remaining are sliceL. Six input bits of the LUTs in sliceM are used to access 64 one-bit locations of a LUTRAM. The number of BRAMs, slices, I/O pins, DSP slices, etc. depends on the FPGA device type and the package size. To make it simple, we choose the one which is mostly used for FPGA-based CAM/TCAM implementation. Table 7 shows the hardware resources available on the Xilinx Virtex-6 FPGA device XC6VLX760. Each DSP slice consists of 25×19 multiplier, an adder, and an accumulator.

7.1. Hardware utilization

FPGAs are enriched with different hardware elements that are ready to make any digital system, i.e., configurable logic blocks, routing matrix, memory elements, dedicated FPU in some cases, etc. How these hardware elements utilize to form any digital system, depends on two factors; synthesizer and programming language instructions. Synthesizer output is, in most cases, predictable, but in some cases, the same code creates two different hardware circuits at different times and different FPGA devices. The programming instructions can be optimized according to the hardware structure of the target FPGA device.

7.2. Power consumption

Power consumption is an intrinsic property of FPGAs. If more power is consumed, more heat is produced, which shortens the life of the target FPGA device. Two types of power are involved in an FPGA design: static power and dynamic power. In CAM designing, dynamic power consumption is important because of the operation of searching and updating the CAM memory. Xilinx Xpower Analyzer provides both static and dynamic power consumed for the synthesized design.

7.3. Speed

The time taken from the input to the output describes the speed (or throughput) of the design. In most cases, hardware utilization has a direct impact on the speed of the design. In general, the larger the hardware circuitry, the lower the speed of the design because of the longer path delays. In some cases, when pipeline registers are introduced, the hardware utilization can be larger, but the speed can still be higher. When pipelining is done to hardware by introducing the pipeline registers, the intermediate values are stored in these pipeline registers, which help to increase the throughput of the design. Xilinx ISE and Vivado provide the timing report, which states the delay in nanoseconds in the form of Worst Negative Slack (WNS).

7.4. Conversion of metrics

We have tried our best to keep the size and device for each design consistent, but in some cases, it is not possible due to design constraints. For example, Xilinx SRL16E-based TCAM design is not scalable up to 512×36 . So we have provided the same results and sizes which are claimed in the document of the said design. Similarly, theoretical normalization can be done for

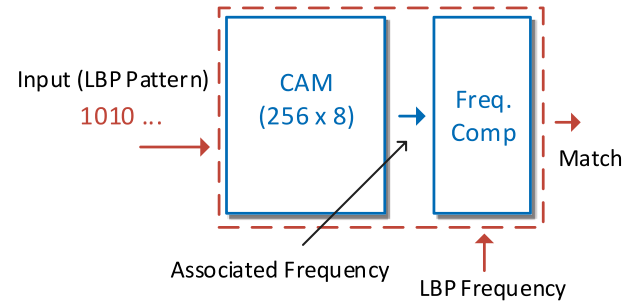


Fig. 15. Using CAM to accelerate the local binary pattern (LBP) based Pattern Recognition on FPGAs. (Freq. Comp: Frequency Comparator, Match: Output to show match or mismatch of the input pattern).

each metric to bring every design on the same page. Still, some experts consider it unfair to compare the consumption of BRAM and LUTRAM or LUTRAM and FFs. So we have provided the actual hardware utilization of each design in Tables 4, 5, and 6. However, the conversion philosophy used in literature is also provided in the form of Equations.

7.4.1. Area calculation

Normalized Area combines the utilization of BRAMs and slices to get the theoretical value of total slices ($Slices'$) which is given as:

$$Slices' = No. \text{ of slices} + No. \text{ of BRAMs}(36k) * 24 \quad (1)$$

7.4.2. Throughput calculation

The device technology has a direct impact on the speed of the digital design, which can be normalized with Virtex-6 FPGA using Eq. (2).

$$Speed' = Speed * \left[\frac{Device \text{ (nm)}}{Virtex 6 \text{ (nm)}} \right] \quad (2)$$

Throughput is the product of speed and the number of bits handled by the CAM/TCAM design in one clock cycle, i.e., the width of CAM/TCAM, given by Eq. (3).

$$Throughput = Speed * W_C \quad (3)$$

7.4.3. Energy calculation

The FPGA developmental tools, in our case, Xilinx ISE and Vivado, provides the power consumption of the digital design in terms of static and dynamic power consumption. It is crucial to link the time taken with the power consumed by design to get a better insight into its performance. Energy is a combination of power and time, which is calculated using Eq. (4) where W_C and D_C are the width and depth of CAM/TCAM, respectively.

$$Energy(fJ/bit/search) = \frac{Power}{Speed * (W_C * D_C)} \quad (4)$$

8. Applications of CAM

With the escalation of cloud computing and data approaching the sea of data, there is an increasing demand for high throughput search engines where FPGAs and FPGA-based CAMs play a significant role [76]. CAMs are used in data compression, translation lookaside buffers (TLBs) in microprocessors, image recognition, content-centric networks [77], and cognitive information processing. The commercial use of TCAM is in-network routers where it classifies the incoming packets against the stored addresses [2, 78,35]. Here we briefly discuss these TCAM applications and a few more.

Neural networks distinguish the input data to either directly store the output or infer based on the training regimen [79]. The network systems provide the correct output when only part of the input pattern is available where CAM can be deployed for such partial searching, which can provide a significant performance improvement.

Intrusion detection [80] enables the network to protect from external attacks. Keyword match processor is the core of a network intrusion detection system (NIDS), which consists of a controller, a CAM, and an encoder that outputs the address. CAM contains all the signatures which are used for authentication of any access to the network. Failing to match the signatures stops access to the network.

Local binary pattern (LBP) is an important technique widely used in computer vision for classification and pattern matching (See Fig. 15). The performance of pattern recognition has shown improvement using an FPGA-based CAM on Xilinx FPGA when the patterns are stored in CAM instead of RAM [81]. They can achieve the match results in 100x less time than the state-of-the-art related systems.

Softcore processors such as Berkley's open-source hardware instruction set architecture (ISA) tend to be the future processors. In these processors, a translation lookaside buffer (TLB) cache reduces the access time and is implemented using CAM/TCAM [82, 83]. FPGAs are highly suitable for the performance evaluation of softcore processors. Furthermore, the FPGA-based CAM/TCAMs are also beneficial in realizing the TLB part [84] for these processors.

9. Conclusion

FPGA-based CAMs are presented not to beat the conventional CAM but rather bring a reconfigurable CAM with the available hardware resources on FPGAs owing to the benefits of SRAM technology and FPGAs. The three major types of FPGA-based CAMs have their trade-offs and their suitability for applications which vary from system to system. SRAM-based TCAM was not implementable on FPGA resources. Partitioning and mapping-the-data-differently made it possible to be implemented on FPGA. Still, the ratio of SRAM cells per CAM bit is higher, which can be reduced by proposing different mapping schemes.

10. Future work and potential research direction

Future FPGA-based CAM can be a simple one-to-one mapping of CAM data into the SRAM blocks, which will be reconfigurable and efficient in terms of hardware resources, power consumption, and throughput. Despite FPGAs having applications where CAMs are not used, there is a possibility that FPGA vendors build a hard-core CAM in future devices to accelerate the searching-based applications. As the world is moving towards artificial intelligence, we see the FPGA-based CAMs changing from “reconfigurable” to “adaptable”, which will be more intelligent and can reconfigure itself according to the need of the system.

The followings are the current issues and potential research direction to develop an optimal FPGA-based CAM/TCAM architecture.

- The primary motivation (or issue) is the non-availability of softcore in modern FPGAs, so an optimal core needs to be proposed.
- BRAM-based implementations suffer from inefficient memory usage and the ratio of SRAM to TCAM bit is high compared to LUTRAM-based CAMs. SRAM/TCAM ratio, in the case of UE-TCAM, is 64.

- LUTRAM RAM-based implementations have less hardware cost, but its performance degrades with the increase in size because of the wider bit-wise ANDing operation. SRAM/TCAM ratio, in the case of D-TCAM, is 14.
- FF-based TCAM designs suffer from high power consumption due to the parallel comparison of every CAM cell.
- Update latency of BRAM-based CAMs is high up to 512 and 256 clock cycles because, for one update operation, all the locations of BRAM need to be updated.
- SRAMs are vulnerable to soft errors, which can be explored to make FPGA-based CAM reliable and secure [74].

CRedit authorship contribution statement

Muhammad Irfan: Conceptualization, Methodology, Formal analysis, Writing – original draft. **Abdurashid Ibrahim Sanka:** Writing – review & editing, Validation, Proofreading. **Zahid Ullah:** Writing – review & editing, Validation. **Ray C.C. Cheung:** Supervision, Resources, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We are grateful to Dr. Derek Pao from City University of Hong Kong, Hong Kong, and Prof. Martin Herbordt from Boston University, USA, for their valuable feedback on this work.

References

- [1] K. Pagiamtzis, A. Sheikholeslami, Content-addressable memory (CAM) circuits and architectures: A tutorial and survey, *IEEE J. Solid-State Circuits* 41 (3) (2006) 712–727.
- [2] R. Karam, R. Puri, S. Ghosh, S. Bhunia, Emerging trends in design and applications of memory-based computing and content-addressable memories, *Proc. IEEE* 103 (8) (2015) 1311–1330.
- [3] J. Hopfield, Brain, computer, and memory, *Eng. Sci.* 46 (1) (1982) 2–7.
- [4] C.-S. Lin, J.-C. Chang, B.-D. Liu, A low-power precomputation-based fully parallel content-addressable memory, *IEEE J. Solid-State Circuits* 38 (4) (2003) 654–662.
- [5] Y.V. Pershin, M. Di Ventra, Neuromorphic, digital, and quantum computation with memory circuit elements, *Proc. IEEE* 100 (6) (2011) 2071–2080.
- [6] J. Liu, H. Gu, W. Wei, Z. Chen, Y. Chen, An efficient shortest path algorithm for content-based routing on 2-D mesh accelerator networks, *Future Gener. Comput. Syst.* 114 (2021) 519–530.
- [7] D.C. Brock, Dudley buck and the computer that never was, *IEEE Spectr.* 51 (4) (2014) 54–69.
- [8] H. Huang, S. Guo, J. Wu, J. Li, Green datapath for TCAM-based software-defined networks, *IEEE Commun. Mag.* 54 (11) (2016) 194–201.
- [9] Q. Guo, X. Guo, Y. Bai, R. Patel, E. Ipek, E.G. Friedman, Resistive ternary content addressable memory systems for data-intensive computing, *IEEE Micro* 35 (5) (2015) 62–71.
- [10] V. Ravikumar, R.N. Mahapatra, TCAM architecture for IP lookup using prefix properties, *IEEE Micro* 24 (2) (2004) 60–69.
- [11] F. Lin, G. Wang, J. Zhou, S. Zhang, X. Yao, High-performance IPv6 address lookup in GPU-accelerated software routers, *J. Netw. Comput. Appl.* 74 (2016) 1–10.
- [12] A. Dorri, S.S. Kanhere, R. Jurdak, MOF-BC: A memory optimized and flexible blockchain for large scale networks, *Future Gener. Comput. Syst.* 92 (2019) 357–373.
- [13] L. Liu, J. Zhu, Z. Li, Y. Lu, Y. Deng, J. Han, S. Yin, S. Wei, A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications, *ACM Comput. Surv.* 52 (6) (2019) 1–39.
- [14] M. Bouaziz, A. Rachedi, A. Belghith, M. Berbineau, S. Al-Ahmadi, EMA-RPL: Energy and mobility aware routing for the internet of mobile things, *Future Gener. Comput. Syst.* 97 (2019) 247–258.
- [15] S. Saraswat, V. Agarwal, H.P. Gupta, R. Mishra, A. Gupta, T. Dutta, Challenges and solutions in software defined networking: A survey, *J. Netw. Comput. Appl.* 141 (2019) 23–58.

- [16] R. Ammendola, A. Biagioni, O. Frezza, W. Geurts, G. Goossens, F.L. Cicero, A. Lonardo, P.S. Paolucci, D. Rossetti, F. Simula, et al., ASIP acceleration for virtual-to-physical address translation on RDMA-enabled FPGA-based network interfaces, *Future Gener. Comput. Syst.* 53 (2015) 109–118.
- [17] K. Vipin, S.A. Fahmy, FPGA dynamic and partial reconfiguration: A survey of architectures, methods, and applications, *ACM Comput. Surv.* 51 (4) (2018) 72.
- [18] W. Choi, K. Jeong, K. Choi, K. Lee, J. Park, Content addressable memory based binarized neural network accelerator using time-domain signal processing, in: *Proceedings of the 55th Annual Design Automation Conference*, ACM, 2018, p. 138.
- [19] S.Q. Zhang, Q. Zhang, A. Tizghadam, B. Park, H. Bannazadeh, R. Boutaba, A. Leon-Garcia, TCAM space-efficient routing in a software defined network, *Comput. Netw.* 125 (2017) 26–40.
- [20] K. Compton, S. Hauck, Reconfigurable computing: A survey of systems and software, *ACM Comput. Surv.* 34 (2) (2002) 171–210.
- [21] T.-S. Chen, D.-Y. Lee, T.-T. Liu, A.-Y. Wu, Dynamic reconfigurable ternary content addressable memory for openflow-compliant low-power packet processing, *IEEE Trans. Circuits Syst. I. Regul. Pap.* 63 (10) (2016) 1661–1672.
- [22] C.-C. Wang, J.-S. Wang, C. Yeh, High-speed and low-power design techniques for TCAM macros, *IEEE J. Solid-State Circuits* 43 (2) (2008) 530–540.
- [23] M.S. Riazi, M. Samragh, F. Koushanfar, Camsure: Secure content-addressable memory for approximate search, *ACM Trans. Embed. Comput. Syst.* 16 (5s) (2017) 136.
- [24] A. Grossi, E. Vianello, C. Zambelli, P. Royer, J.-P. Noel, B. Giraud, L. Perniola, P. Olivo, E. Nowak, Experimental investigation of 4-kb RRAM arrays programming conditions suitable for TCAM, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 26 (12) (2018) 2599–2607.
- [25] D. Jothi, R. Sivakumar, Design and analysis of power efficient binary content addressable memory (PEBCAM) core cells, *Circuits Systems Signal Process.* 37 (4) (2018) 1422–1451.
- [26] S. Mishra, A. Dandapat, Energy-efficient adaptive match-line controller for large-scale associative storage, *IEEE Trans. Circuits Syst. II Express Briefs* 64 (6) (2016) 710–714.
- [27] U. Dhawan, A. DeHon, Area-efficient near-associative memories on FPGAs, *ACM Trans. Reconfigurable Technol. Syst.* 7 (4) (2015) 30.
- [28] S. Dharmapuri, P. Krishnamurthy, D.E. Taylor, Longest prefix matching using bloom filters, *IEEE/ACM Trans. Netw.* 14 (2) (2006) 397–409.
- [29] K.-C. Woo, B.-D. Yang, Low-area TCAM using a don't care reduction scheme, *IEEE J. Solid-State Circuits* 53 (8) (2018) 2427–2433.
- [30] H.E. Yantir, A.M. Eltawil, F.J. Kurdahi, Approximate memristive in-memory computing, *ACM Trans. Embed. Comput. Syst.* 16 (5s) (2017) 1–18.
- [31] M. Irfan, A. Ahmad, Impact of initialization on gate-based area efficient ternary content-addressable memory, in: *2018 International Conference on Computing, Electronics & Communications Engineering, ICCECE, IEEE*, 2018, pp. 328–332.
- [32] I. Maity, A. Mondal, S. Misra, C. Mandal, CURE: Consistent update with redundancy reduction in SDN, *IEEE Trans. Commun.* 66 (9) (2018) 3974–3981.
- [33] H. Liu, Routing table compaction in ternary CAM, *IEEE Micro* 22 (1) (2002) 58–64.
- [34] C.A. Zerbini, J.M. Finochietto, Performance evaluation of packet classification on FPGA-based TCAM emulation architectures, in: *Global Communications Conference, GLOBECOM, 2012 IEEE*, IEEE, 2012, pp. 2766–2771.
- [35] H. Chen, Y. Chen, D.H. Summerville, A survey on the application of FPGAs for network infrastructure security, *IEEE Commun. Surv. Tutor.* 13 (4) (2010) 541–561.
- [36] S.M.S. Trimberger, Three ages of FPGAs: A retrospective on the first thirty years of FPGA technology: This paper reflects on how Moore's law has driven the design of FPGAs through three epochs: The age of invention, the age of expansion, and the age of accumulation, *IEEE Solid-State Circuits Mag.* 10 (2) (2018) 16–29.
- [37] A.M. Abdelhadi, G.G. Lemieux, L. Shannon, Modular block-RAM-based longest-prefix match ternary content-addressable memories, in: *2018 28th International Conference on Field Programmable Logic and Applications, FPL, IEEE*, 2018, pp. 243–2437.
- [38] T. Stimpfling, N. Bélanger, J.P. Langlois, Y. Savaria, SHIP: A scalable high-performance IPv6 lookup algorithm that exploits prefix characteristics, *IEEE/ACM Trans. Netw.* 27 (4) (2019) 1529–1542.
- [39] M. Zackriya, H.M. Kittur, Precharge-free, low-power content-addressable memory, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 24 (8) (2016) 2614–2621.
- [40] J.-L. Brelet, Methods for implementing CAM functions using dual-port RAM, *US Patent 6,353,332* (Mar. 5 2002).
- [41] J.-L. Brelet, Using block RAM for high performance read/write CAMs, 2000, Xilinx Inc., Application Notes 204.
- [42] J.-L. Brelet, L. Gopalakrishnan, Using virtex-ii block RAM for high performance read/write CAMs, 2002, Xilinx Application Note XAPP260.
- [43] A. Corp, Implementing high-speed search applications with altera cam, 2001, Application Note 119.
- [44] M. Somasundaram, Circuits to generate a sequential index for an input number in a pre-defined list of numbers, *US Patent 7,155,563* (Dec. 26 2006).
- [45] Z. Ullah, K. Ilgon, S. Baeg, Hybrid partitioned SRAM-based ternary content addressable memory, *IEEE Trans. Circuits Syst. I. Regul. Pap.* 59 (12) (2012) 2969–2979.
- [46] Z. Ullah, M.K. Jaiswal, R.C. Cheung, Z-TCAM: An SRAM-based architecture for TCAM, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 23 (2) (2015) 402–406.
- [47] Z. Ullah, M.K. Jaiswal, R.C. Cheung, E-TCAM: An efficient SRAM-based architecture for TCAM, *Circuits Systems Signal Process.* 33 (10) (2014) 3123–3144.
- [48] Z. Ullah, M.K. Jaiswal, R.C. Cheung, H.K. So, UE-TCAM: An ultra efficient SRAM-based TCAM, in: *TENCON 2015-2015 IEEE Region 10 Conference*, IEEE, 2015, pp. 1–6.
- [49] K. Locke, Parameterizable content-addressable memory, 2011, Xilinx Application Note XAPP1151.
- [50] Xilinx, Ternary Content Addressable Memory (TCAM) Search IP for SDNet, Xilinx Product Guide PG190, Xilinx, San Jose, USA, 2017.
- [51] M. Somasundaram, Memory and power efficient mechanism for fast table lookup, *US Patent 7,296,113* (Nov. 13 2007).
- [52] W. Yu, S. Sivakumar, D. Pao, Pseudo-TCAM: SRAM-based architecture for packet classification in one memory access, *IEEE Netw. Lett.* 1 (2) (2019) 89–92.
- [53] Z. Qian, M. Margala, Low power RAM-based hierarchical CAM on FPGA, in: *ReConfigurable Computing and FPGAs, ReConFig, 2014 International Conference on*, IEEE, 2014, pp. 1–4.
- [54] A. Ahmed, K. Park, S. Baeg, Resource-Efficient SRAM-Based ternary content addressable memory, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 25 (4) (2017) 1583–1587.
- [55] I. Ullah, Z. Ullah, J.-A. Lee, EE-TCAM: An energy-efficient SRAM-based TCAM on FPGA, *Electronics* 7 (9) (2018) 186.
- [56] W. Jiang, V.K. Prasanna, Scalable packet classification on FPGA, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 20 (9) (2011) 1668–1680.
- [57] W. Jiang, Scalable ternary content addressable memory implementation using FPGAs, in: *Proceedings of the Ninth ACM/IEEE Symposium on Architectures for Networking and Communications Systems*, IEEE Press, 2013, pp. 71–82.
- [58] I. Ullah, Z. Ullah, J.-A. Lee, Efficient TCAM design based on multipumping-enabled multiported SRAM on FPGA, *IEEE Access* 6 (2018) 19940–19947.
- [59] R.E. Corporation, Renesas' Network Search Engine (R8A20410BG), URL <https://www.renesas.com/us/en/products/memory/application-specific-memory/network-search-engine/device/R8A20410BG-G.html>.
- [60] R. Oda, T. Yamada, T. Ikegaya, T. Tsumura, H. Matsuo, Y. Nakashima, Input entry integration for an auto-memoization processor, in: *2011 Second International Conference on Networking and Computing*, IEEE, 2011, pp. 179–185.
- [61] I. Ullah, Z. Ullah, U. Afzaal, J.-A. Lee, DURE: An energy-and resource-efficient TCAM architecture for FPGAs with dynamic updates, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* (2019).
- [62] P. Reviriego, A. Ullah, S. Pontarelli, PR-TCAM: Efficient TCAM emulation on xilinx FPGAs using partial reconfiguration, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* (2019).
- [63] E. Rossi, M. Damschen, L. Bauer, G. Buttazzo, J. Henkel, Preemption of the partial reconfiguration process to enable real-time computing with FPGAs, *ACM Trans. Reconfigurable Technol. Syst.* 11 (2) (2018) 10.
- [64] M.S. Reorda, L. Sterpone, A. Ullah, An error-detection and self-repairing method for dynamically and partially reconfigurable systems, *IEEE Trans. Comput.* 66 (6) (2016) 1022–1033.
- [65] M. Irfan, Z. Ullah, R. C.C. Cheung, Zi-CAM: A power and resource efficient binary content-addressable memory on FPGAs, *Electronics* 8 (5) (2019) 584.
- [66] M. Irfan, Z. Ullah, R.C. Cheung, D-TCAM: A high-performance distributed RAM based TCAM architecture on FPGAs, *IEEE Access* 7 (2019) 96060–96069.
- [67] A. Zahir, S.K. Khattak, A. Ullah, P. Reviriego, F.B. Muslim, W. Ahmad, FracTCAM: Fracturable LUTRAM-based TCAM emulation on Xilinx FPGAs, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 28 (12) (2020) 2726–2730.
- [68] F. Syed, Z. Ullah, Updating algorithm for SRAM-based TCAM and its implementation on FPGA, *Int. J. Comput. Sci. Inf. Secur.* 15 (1) (2017) 116.
- [69] F. Syed, Z. Ullah, M.K. Jaiswal, Fast content updating algorithm for an SRAM-based TCAM on FPGA, *IEEE Embedded Syst. Lett.* 10 (3) (2017) 73–76.
- [70] Z. Ullah, LH-CAM: Logic-based higher performance binary CAM architecture on FPGA, *IEEE Embedded Syst. Lett.* 9 (2) (2017) 29–32.
- [71] M. Irfan, Z. Ullah, G-AETCAM: Gate-based area-efficient ternary content-addressable memory on FPGA, *IEEE Access* 5 (2017) 20785–20790.

- [72] H. Mahmood, Z. Ullah, O. Mujahid, I. Ullah, A. Hafeez, Beyond the limits of typical strategies: Resources efficient FPGA-based TCAM, *IEEE Embedded Syst. Lett.* (2018).
- [73] P. Mahoney, Y. Savaria, G. Bois, P. Plante, Performance characterization for the implementation of content addressable memories based on parallel hashing memories, in: *Transactions on High-Performance Embedded Architectures and Compilers II*, Springer, 2009, pp. 307–325.
- [74] P. Reviriego, S. Pontarelli, A. Ullah, Error detection and correction in SRAM emulated TCAMs, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 27 (2) (2019) 486–490.
- [75] Xilinx, Virtex-6 FPGA family, URL <https://www.xilinx.com/support/documentation-navigation/silicon-devices/fpga/virtex-6.html>.
- [76] S. Khoram, Y. Zha, J. Zhang, J. Li, Challenges and opportunities: From near-memory computing to in-memory computing, in: *Proceedings of the 2017 ACM on International Symposium on Physical Design*, ACM, 2017, pp. 43–46.
- [77] H. Yang, X. Wang, C. Yang, X. Cong, Y. Zhang, Securing content-centric networks with content-based encryption, *J. Netw. Comput. Appl.* 128 (2019) 21–32.
- [78] W. Wu, J. Shi, L. Zuo, B. Shi, Power-efficient TCAMs for bursty access patterns, *IEEE Micro* 25 (4) (2005) 64–72.
- [79] M. Imani, D. Peroni, Y. Kim, A. Rahimi, T. Rosing, Efficient neural network acceleration on gpgpu using content addressable memory, in: *Proceedings of the Conference on Design, Automation & Test in Europe*, European Design and Automation Association, 2017, pp. 1026–1031.
- [80] C.E. Graves, C. Li, X. Sheng, W. Ma, S.R. Chalamalasetti, D. Miller, J.S. Ignowski, B. Buchanan, L. Zheng, S.-T. Lam, et al., Memristor TCAMs accelerate regular expression matching for network intrusion detection, *IEEE Trans. Nanotechnol.* (2019).
- [81] O. Mujahid, Z. Ullah, H. Mahmood, A. Hafeez, Fast pattern recognition through an LBP driven CAM on FPGA, *IEEE Access* 6 (2018) 39525–39531.
- [82] Y. Zhang, L. Zhou, Y. Makris, Hardware-based real-time workload forensics via frame-level TLB profiling, in: *2019 IEEE 37th VLSI Test Symposium, VTS*, IEEE, 2019, pp. 1–6.
- [83] S. Mittal, A survey of techniques for architecting TLBs, *Concurr. Comput.: Pract. Exper.* 29 (10) (2017) e4061.
- [84] N.C. Papadopoulos, V. Karakostas, K. Nikas, N. Koziris, D.N. Pnevmatikatos, A configurable TLB hierarchy for the RISC-V architecture, in: *2020 30th International Conference on Field-Programmable Logic and Applications, FPL*, IEEE, 2020, pp. 85–90.



Muhammad Irfan received the B.Sc. degree in electrical engineering from University of Engineering and Technology, Peshawar, Pakistan, in 2013, and the M.S. degree in electrical engineering from Lahore University of Management Sciences, Lahore, Pakistan, in 2016, respectively.

He has taught for two years in the Department of Electrical Engineering, CECOS University of IT & Emerging Sciences, Peshawar, Pakistan, and is currently on leave from there. He is doing Ph.D. in Electrical Engineering at City University of Hong Kong, Hong Kong. His research interests include FPGA-based digital systems designs, low power computer architectures, and memory design.



Abdurrahid Ibrahim Sanka received B.Eng. Electrical degree from Bayero University, Kano, Nigeria, in 2011. He got MSc. degree in Embedded Microelectronics and Wireless Systems from Coventry University, United Kingdom, in 2014. He has been working with the Bayero University, Kano as a lecturer.

Currently, he is working towards Ph.D. degree with the department of Electrical Engineering, City University of Hong Kong. His research interests include digital systems design, blockchain technology, and network computing and security,



Zahid Ullah received the B.Sc. degree (Hons.) in computer system engineering from the University of Engineering and Technology, Peshawar, Pakistan in 2006, the M.S. degree in electronic, electrical, control, and instrumentation engineering from Hanyang University, South Korea in 2010, and the Ph.D. degree in electronic engineering from the City University of Hong Kong, Hong Kong in 2014. He is currently serving as Assistant Professor in the Department of Electrical & Computer Engineering, Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology, Haripur, Pakistan.

He has previously served as Associate Professor & Chairman with the Department of Electrical Engineering, CECOS University of IT & Emerging Sciences, Peshawar, Pakistan.



Ray C.C. Cheung received the BEng and MPhil degrees in computer engineering and computer science and engineering at the Chinese University of Hong Kong (CUHK), Hong Kong, in 1999 and 2001, respectively, and the Ph.D. degree and DIC in computing at Imperial College London, London, United Kingdom, in 2007. After completing the Ph.D. work, he received the Hong Kong Croucher Foundation Fellowship for his postdoctoral study in the Electrical Engineering Department, University of California, Los Angeles (UCLA). In 2009, he worked as a visiting research fellow in the Department

of Electrical Engineering, Princeton University, Princeton, NJ. Currently, he is an associate professor in the Department of Electronic Engineering, City University of Hong Kong (CityU). He is the author of more than 150 journal and conference papers. His research team, CityU Architecture Lab for Arithmetic and Security (CALAS), focuses on the following research topics: reconfigurable trusted computing, applied cryptography, and high-performance biomedical VLSI designs. He is a member of the IEEE.